# AI-Proactive Road Safety System

## 1. Project Planning & Management :

### Project Overview

The AI-Proactive Road Safety System is a fully simulated predictive safety solution designed to anticipate and prevent road accidents **2–3 seconds before they occur**. By integrating CARLA (photorealistic driving simulator), SUMO (traffic flow simulator), and advanced AI frameworks (TensorFlow, PyTorch, YOLO, Graph Neural Networks), the system creates a virtual prototype that demonstrates how AI can detect risks and take preventive action ,all without physical vehicles or dangerous real-world trials. This approach enables safe, cost-efficient experimentation while delivering insights into accident prevention strategies.

---

### Objectives

- **Accident Anticipation**: Train AI models to predict collisions seconds before impact, outperforming human reaction times.
- **Risk Detection**: Identify high-risk behaviors such as distracted pedestrians, aggressive drivers, and chain-reaction braking.
- **Data Integration**: Fuse macro-level traffic flow (SUMO) with micro-level sensor realism (CARLA) to build a comprehensive dataset.
- **Model Development**: Implement YOLO for perception, GNNs for intent prediction, and risk scoring models in TensorFlow/PyTorch.
- **Cloud Deployment**: Deliver a scalable, containerized accident prediction service via Azure ML with REST API endpoints.
- **Stakeholder Visibility**: Provide dashboards and monitoring tools to visualize risks, predictions, and accident prevention statistics.

---

### Scope

- **Inclusions**:
    - Data collection from CARLA and SUMO simulations.
    - Preprocessing, cleaning, and exploratory data analysis of accident scenarios.
    - Development and evaluation of predictive AI models.

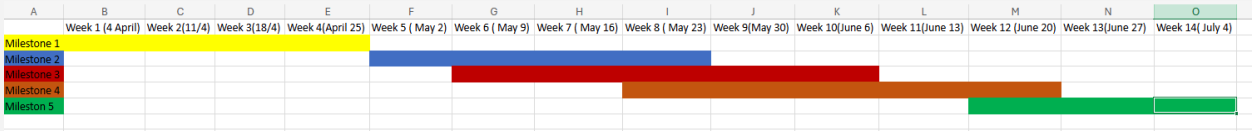
    - Cloud deployment with Docker and Azure ML pipelines.
    - Interactive dashboards for monitoring and reporting.
- **Exclusions**:
    - Hardware integration with real vehicles (focus remains on simulation and cloud services).
    - Non-traffic domains outside road safety.
- **Boundaries**:

    - The project is limited to simulated environments with potential extension to real-world deployment in future phases.
    - Deliverables include cleaned datasets, trained models, cloud services, dashboards, and final documentation with a roadmap for real-world integration.

---

<u>Project Plan</u>

## Timeline (Gantt chart):




## Milestones and deliverables:

### Milestone 1: Data Collection, Preprocessing, and EDA

- **Week 1 (Apr 4)** → Generate SUMO
- **Week 2 (Apr 11)** →CARLA scenarios, collect sensor data.
- **Week 3 (Apr 18)** →Continue CARLA, preprocess (normalize, clean, structure).
- **Week 4 (Apr 25)** → Perform EDA, identify accident patterns.
- **Deliverables**: Cleaned dataset, preprocessing pipeline, EDA report.

### Milestone 2: Model Development & Evaluation

- **Week 5 (May 2)** → Start YOLO training for object detection.
- **Week 6 (May 9)** → Implement GNN for intent prediction.
- **Week 7 (May 16)** → Train risk scoring models in TensorFlow/PyTorch.
- **Week 8 (May 23)** → Evaluate models, compare AI vs. human reaction times.
- **Deliverables**: Trained predictive AI models, evaluation report.

### Milestone 3: Advanced Techniques & Cloud Deployment

- **Week 6 (May 9)** → Begin Docker containerization prep.
- **Week 7 (May 16)** → Deploy via Azure ML, set up REST API endpoints.
- **Week 8 (May 23)** → Integrate SUMO + CARLA outputs with Azure pipelines.
- **Week 9 (May 30)** → Demonstrate live scenarios (pedestrian, aggressive driver, chain reaction braking).
- **Week 10 (June 6)** → Finalize deployment testing.
- **Deliverables**: Deployed accident predictor service, REST API integration

**Milestone 4: MLOps, Monitoring & Dashboard**

- **Week 8 (May 23)** → Start dashboard prototyping (Power BI/Streamlit).
- **Week 9 (May 30)** → Implement MLflow tracking, drift detection.
- **Week 10 (June 6)** → Build retraining pipelines.
- **Week 11 (June 13)** → Risk heatmaps, decision timelines.
- **Week 12 (June 20)** → Accident prevention statistics, configure alerts.
- **Week 13 (June 27)** → Finalize dashboards and monitoring pipeline.
- **Deliverables**: Monitoring pipeline, interactive dashboard, MLOps documentation.

**Milestone 5: Final Documentation & Presentation**

- **Week 12 (June 20)** → Begin drafting technical report and presentation slides.
- **Week 13 (June 27)** → Polish report, integrate dashboards, prepare demo.
- **Week 14 (July 4)** → Final presentation rehearsal.
- **Deadline: July 10, 2026** → Submit final report and presentation.
- **Deliverables**: Final report, presentation deck, Demo (if present).

**Resources Allocation:**

- **Software & Tools**:
  - SUMO, CARLA → Simulation and scenario generation.
  - TensorFlow, PyTorch → Model training and evaluation.
  - Docker → Containerization for deployment.
  - MLflow → Experiment tracking and retraining pipelines.
  - Power BI / Streamlit → Visualization and dashboards.
- **Cloud Infrastructure (Microsoft Azure)**:
  - **Model Training & Deployment** → Use Azure ML to train models and deploy them as REST APIs.
  - **Containerization & Pipelines** → Host Docker containers and integrate SUMO + CARLA outputs with Azure pipelines for real-time testing.
  - **Monitoring & Automation** → Track experiments, detect drift, and trigger retraining pipelines using Azure ML services.
  - **Integration with Streamlit** → Streamlit dashboards connect to Azure endpoints, visualizing predictions and accident risk scores in real time.

## 3. Task assignment and roles:

For every milestone, the project team of six members will contribute equally. Each milestone is divided into six parallel tasks, ensuring balanced workload and continuous collaboration. Roles are not fixed to individuals; instead, tasks are distributed dynamically based on availability and expertise.

**<u>Finished Milestone Task Distribution:</u>**

Milestone one:

| Name | Role |
|---|---|
| Hania Mohamed | SUMO traffic simulation |
| Menna Mohamed | CARLA scenarios, collect sensor data. |
| Mahrael Raafat | preprocess (normalize, clean, structure). |
| Maryam Ahmed | preprocess (normalize, clean, structure). |
| Bassant tarik | preprocess (normalize, clean, structure). |
| Rawan Mosaad | EDA |

## 4.Risk Assessment & Mitigation Plan:

**1. Data Risks**

- **Risk**: Imbalanced scenarios (e.g., too few pedestrian or aggressive driver cases).
- **Mitigation**: Apply oversampling/augmentation, generate additional CARLA/SUMO scenarios, and balance datasets before training.

**2. Model Risks**

- **Risk**: Overfitting of YOLO/GNN models on limited scenarios.
- **Mitigation**: Use regularization, dropout, cross-validation, and monitor performance with unseen test cases.

**3. Deployment Risks**

- **Risk**: Cloud deployment issues (Azure ML resource limits, container errors).
- **Mitigation**: Test locally with Docker before cloud deployment, use lightweight VM sizes, and monitor resource usage.

**KPIs:**

**Response Time** → REST API returns predictions within **≤200 ms** per request.
**System Uptime** → Accident predictor service available **≥95%** during testing/demo.
**Model Accuracy** → YOLO mAP **≥85%**, GNN intent prediction **≥80%**, Risk scoring ROC-AUC **≥0.85**.

---

# Requirement Gatherings:

## Stakeholder Analysis :

- **Drivers (Human Operators)**
  - **Needs**: Early accident warnings, clear dashboard alerts, and minimal distraction while driving.
  - **Concerns**: Accuracy of predictions, response time, and trust in system reliability.
- **Pedestrians & Road Users**
  - **Needs**: Safer crossings, reduced risk from aggressive drivers or chain-reaction braking.
  - **Concerns**: System must detect and anticipate pedestrian behavior accurately.
- **Transport Authorities / Regulators**
  - **Needs**: Scalable accident prediction system for monitoring traffic safety.
  - **Concerns**: Compliance with road safety standards, integration with existing infrastructure.

## User Stories & Use Cases:

User Stories

- As a driver, I want real-time accident risk alerts so that I can react quickly and avoid collisions.
- As a pedestrian, I want the system to detect crossing risks so that I can safely navigate intersections.

- As a transport authority, I want dashboards showing accident probability and risk heatmaps so that I can monitor traffic safety trends.

## Use Cases

1. **Accident Prediction**
    - Input: SUMO + CARLA metadata (nodes, edges).
    - Process: YOLO detects objects → GNN predicts intent → Risk scoring model evaluates collision likelihood.
    - Output: Real-time risk alert to driver dashboard.
2. **Scenario Demonstration**
    - Pedestrian crossing, aggressive driver, chain reaction braking.
    - System predicts risk and displays alerts in dashboard.
3. **Monitoring & MLOps**
    - MLflow tracks experiments, drift detection monitors data changes.
    - Retraining pipelines update models when accuracy drops.
    - Dashboard visualizes risk heatmaps, decision timelines, and accident statistics.
4. **Deployment & Accessibility**
    - Models containerized with Docker, deployed on Azure ML.
    - REST API provides predictions to external applications.
    - Streamlit dashboards allow stakeholders to interact with results.

## Functional Requirements:

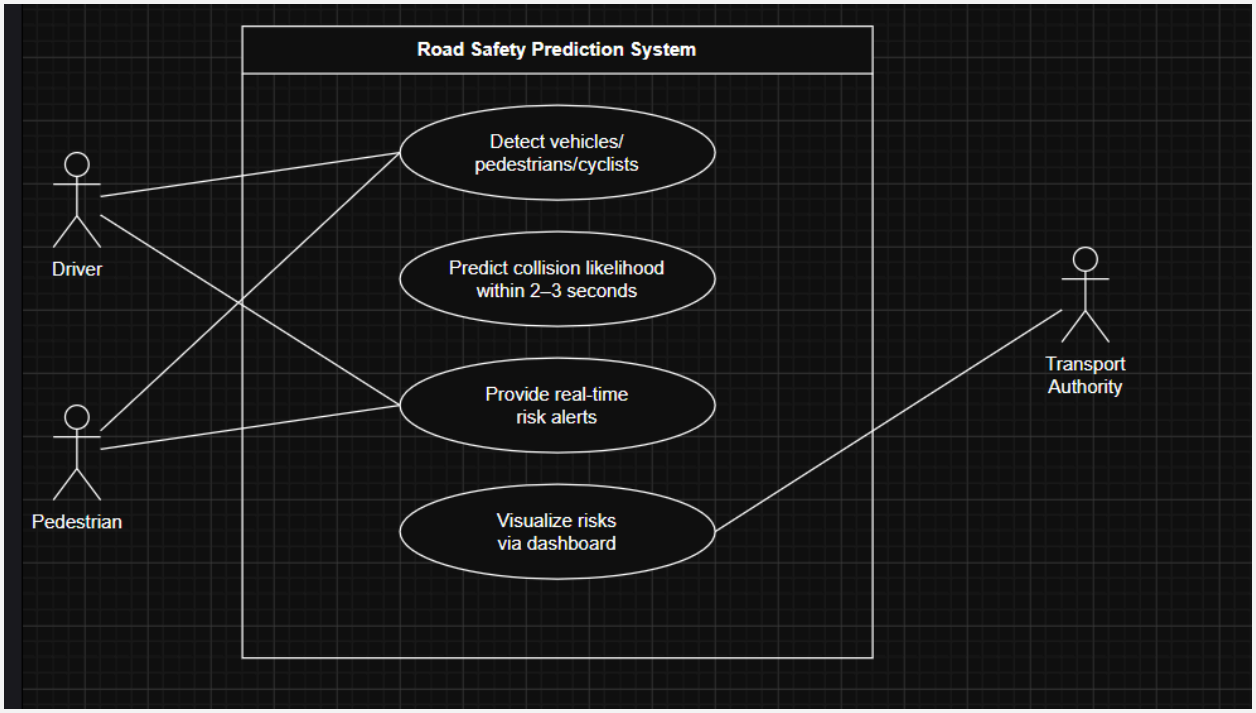
**1**: The system must detect and represent vehicles and pedestrians using **CARLA metadata generated from SUMO traffic simulations**, rather than raw video frames.
**2**: The system must predict collision likelihood within a **2–3 second safety buffer**, leveraging risk scores and actor interactions from metadata.
**3**: The system must provide **real-time risk visualization** through dashboards (heatmaps, alerts, decision timelines).
**5**: The system must deliver accident risk alerts via **REST API endpoints** for integration with dashboards or external applications.

## Non-functional Requirements:

**Performance**: Prediction latency must remain **under 500 ms** per request to allow preventive action.
**Scalability**: Must support **cloud deployment via Azure ML** for global accessibility.
**Reliability**: Service must maintain **≥95% uptime** during testing and demonstrations.
**Usability**: Dashboards must achieve **≥80% positive feedback** from test users.
**Maintainability**: Models must be retrainable within **≤2 hours** when drift is detected, using automated pipelines.

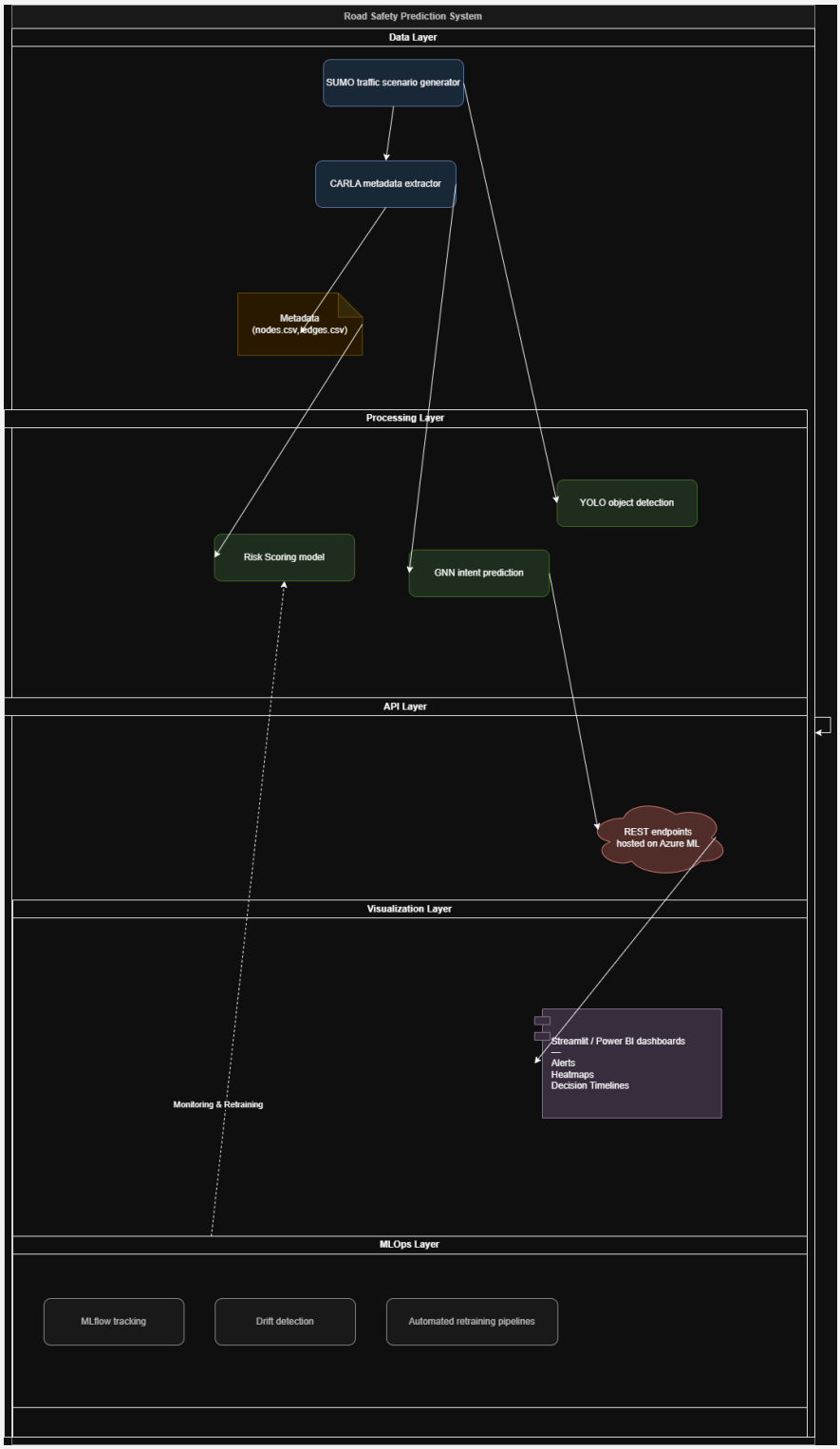
# System Analysis & Design:

## Use Case Diagram:




**Driver** → Receives real-time alerts and dashboard notifications.
**Pedestrian** → Detected as an external entity in scenarios.
**Transport Authority** → Accesses dashboards for monitoring accident statistics and risk heatmaps.

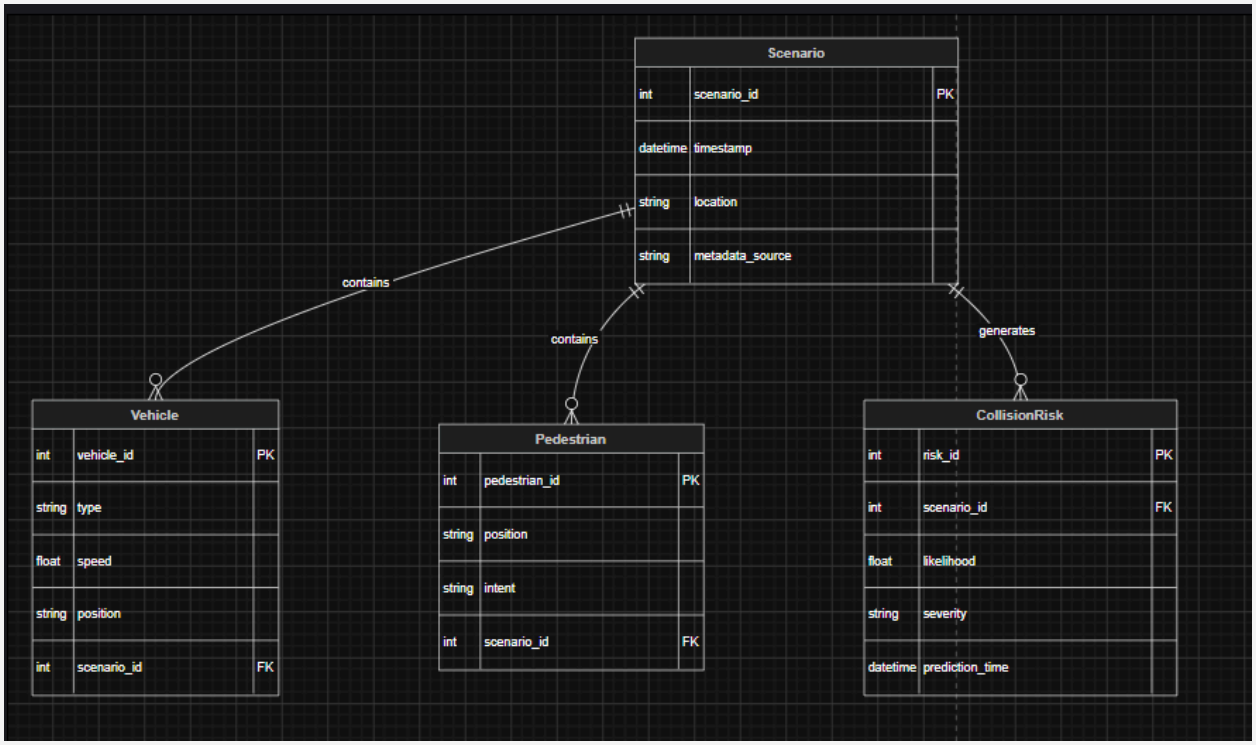
## Software Architecture



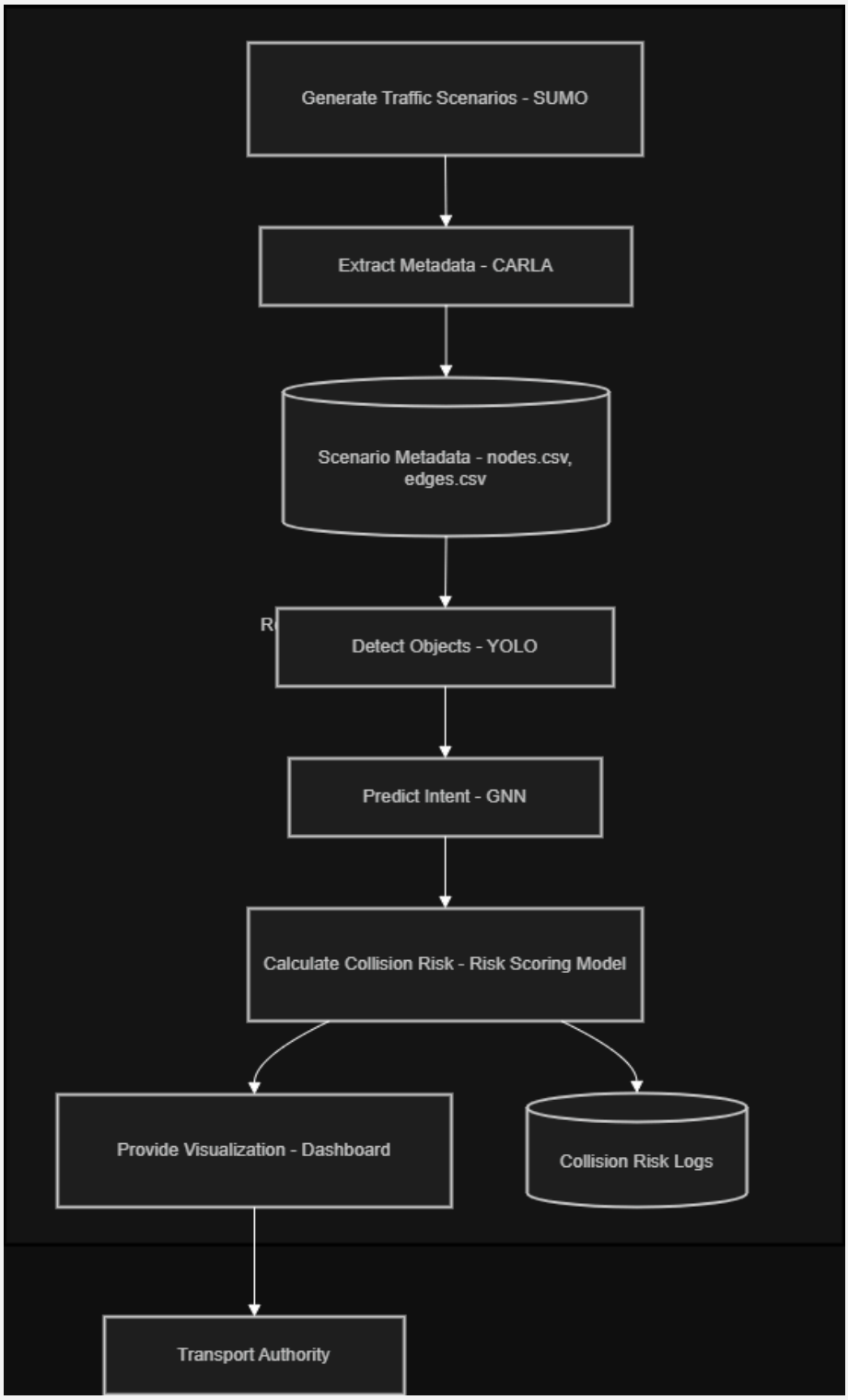

**Data Layer** → SUMO traffic scenario generator + CARLA metadata extractor feeding into structured graph data (nodes/edges).

**Processing Layer** → YOLO for detection, GNN for intent prediction, Risk Scoring model for collision likelihood.
**API Layer** → REST endpoints hosted on Azure ML to expose predictions externally.
**Visualization Layer** → Streamlit/Power BI dashboards for alerts, heatmaps, and decision timelines.
**MLOps Layer** → MLflow tracking, drift detection, and automated retraining pipelines ensuring reliability.

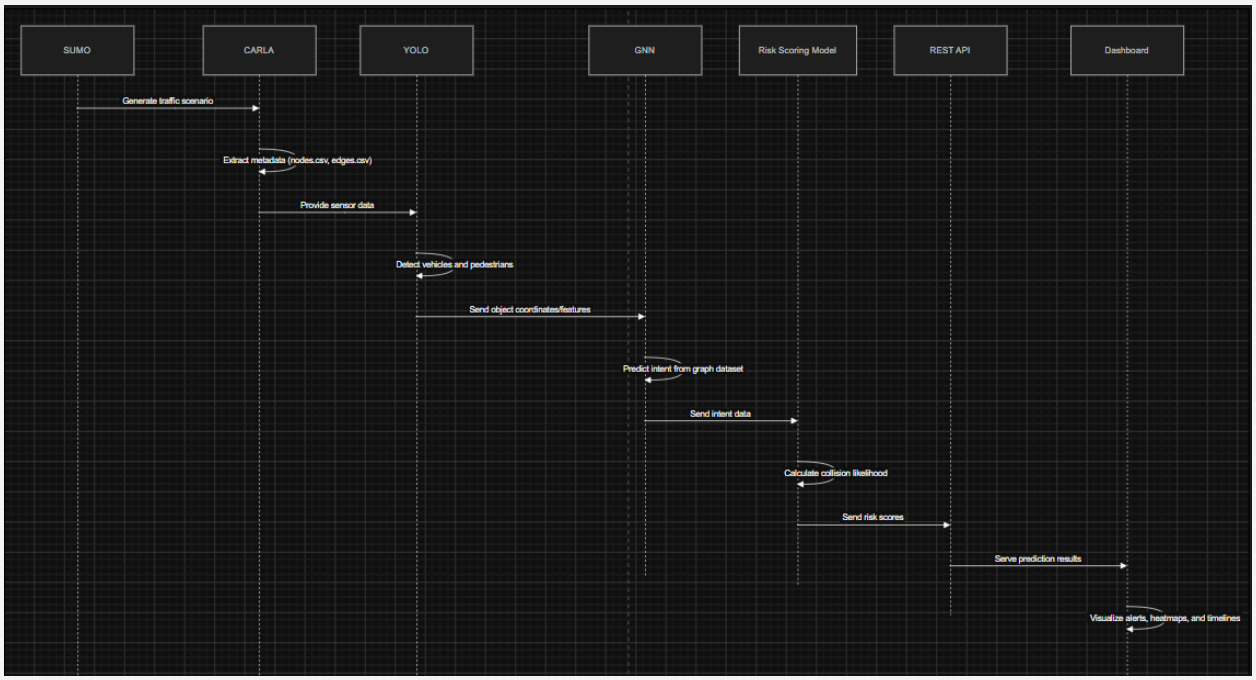
## Database Design & Data Modeling



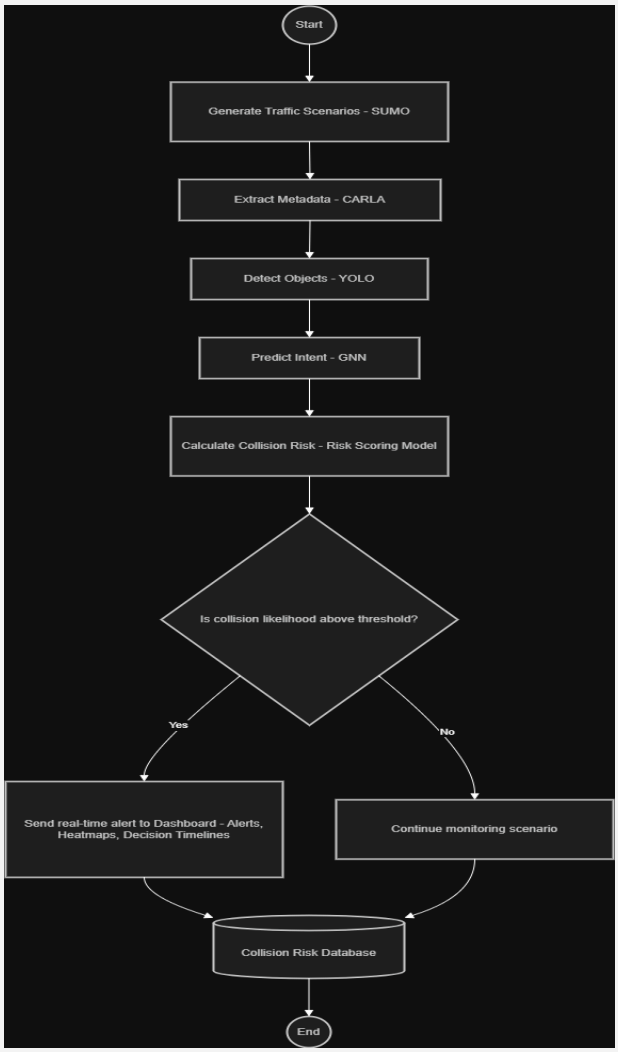
## Context Diagram:



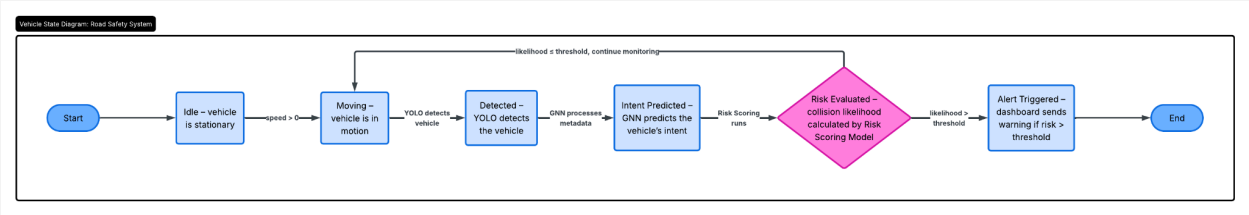
## Sequence Diagram:



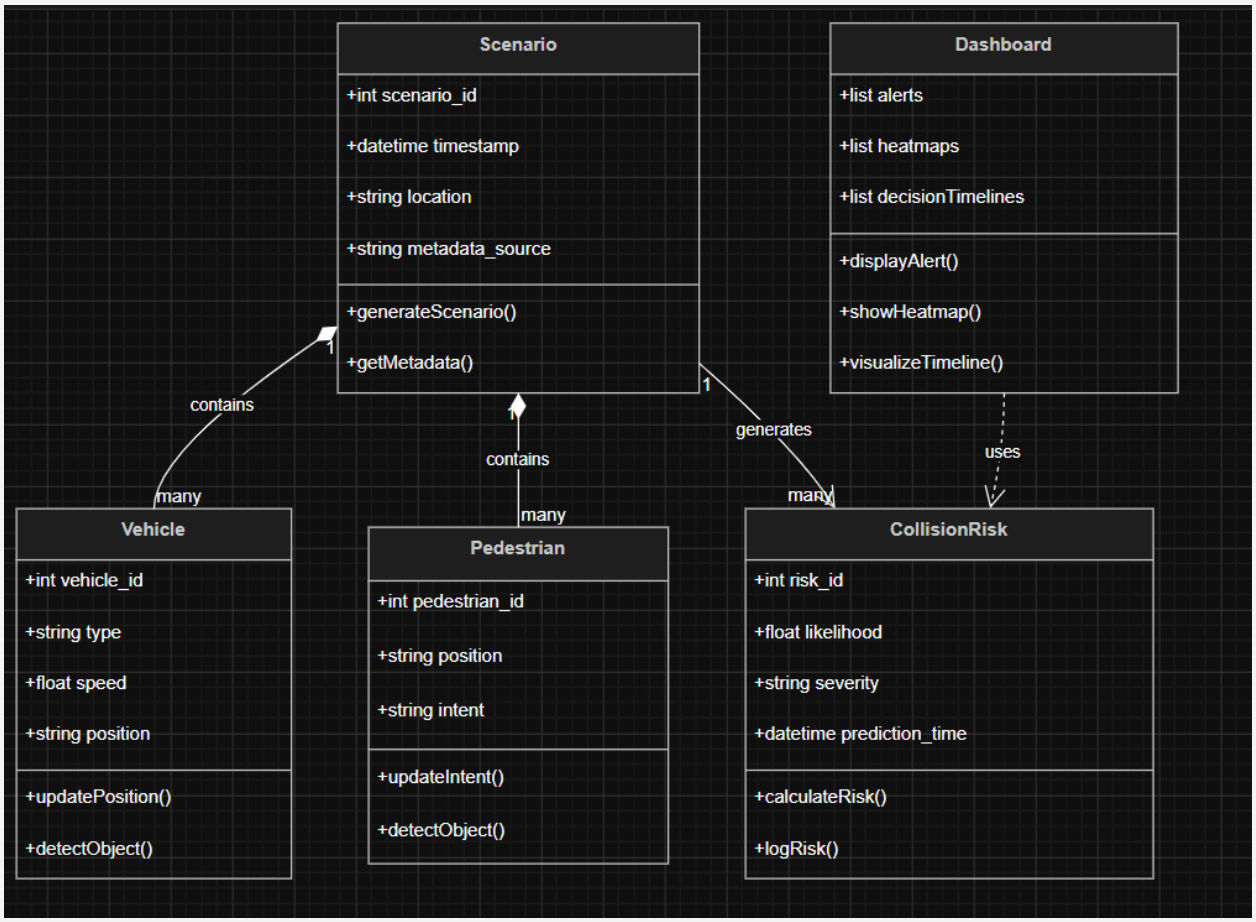
## Activity Diagram:




## State Diagram:



## Class Diagram




## UI/UX:

The UI/UX phase has not yet started, as the team has just completed the generation stage. Once initiated, design will follow these principles:

- **Clarity & Simplicity** :Risk alerts must be easy to read and act upon.
- **Consistency** :Unified colors, typography, and layouts across dashboards.
- **Accessibility** :Compliance with usability and accessibility standards.
- **Responsiveness** :Interfaces adaptable to desktop, tablet, and mobile.
- **Visual Style** :Intuitive color coding (green = safe, yellow = caution, red = high risk), clear icons, and legible fonts.
- **Tools** :Streamlit for dashboards, Figma for mockups, Power BI for advanced reporting.

**Status:** Guidelines are defined, but design work will begin after preprocessing and model development milestones.

## System deployment and integration:

**Backend (ML & Processing)**

- **Azure ML** → Model training, deployment, and inference (YOLO, GNN, Risk Scoring).
- **Python** → Core scripts for preprocessing, simulation integration, and risk calculation.
- **MLflow** → Model tracking and performance monitoring.
- **Simulation Tools** → SUMO (traffic scenarios), CARLA (metadata extraction).

**Frontend (Visualization & Interaction)**

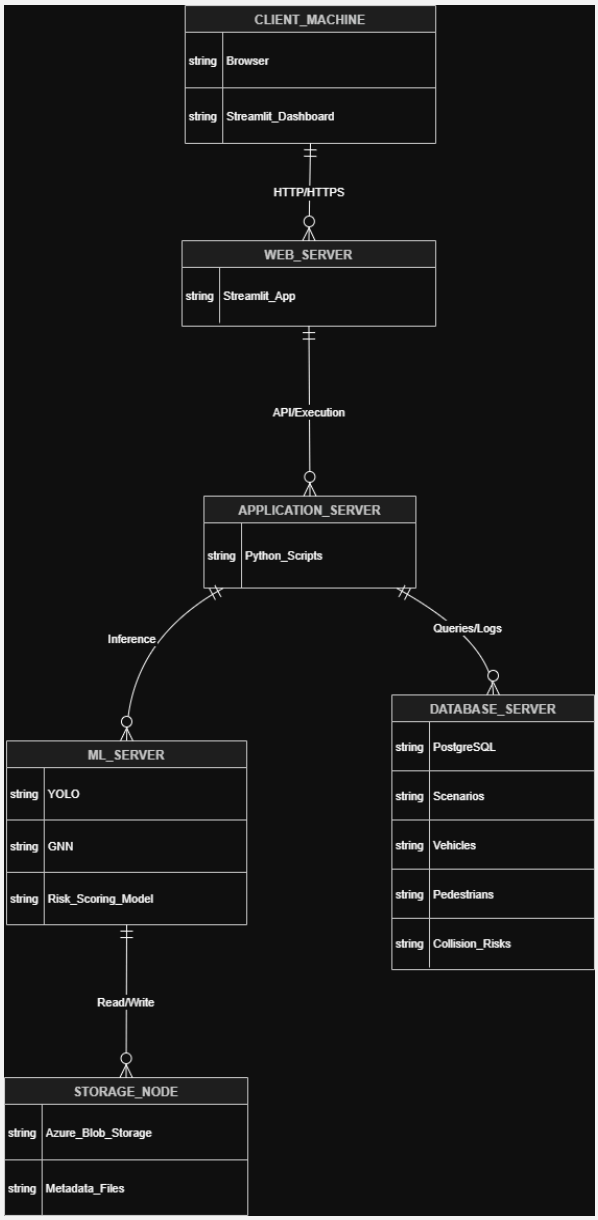
- **Streamlit** → Directly handles both UI and backend logic for dashboards, alerts, and heatmaps.

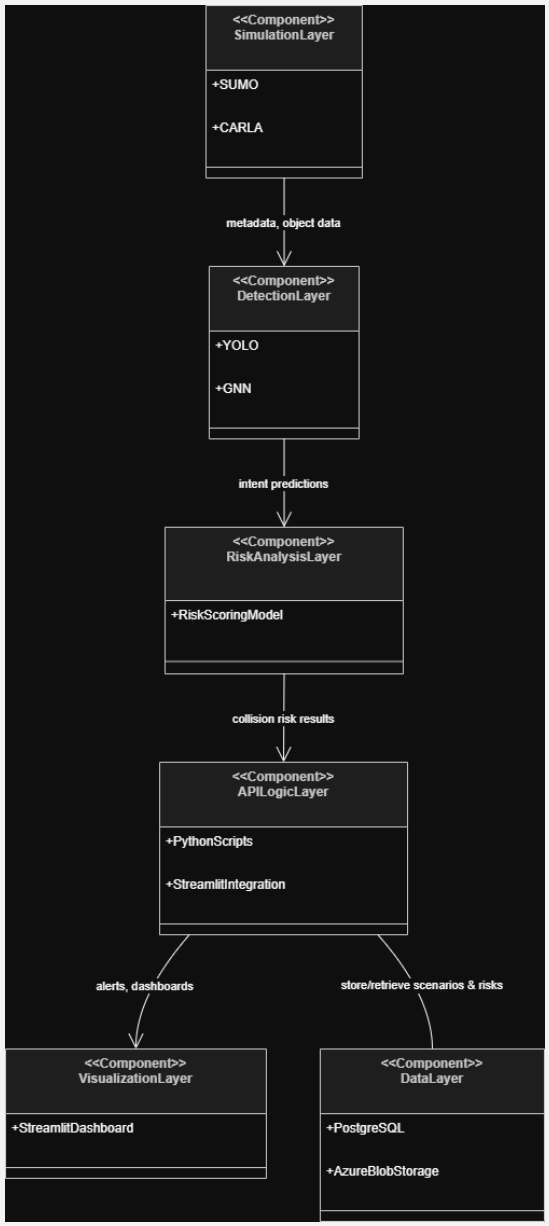
**Database & Storage**

- **PostgreSQL** → Structured storage of scenarios, vehicles, pedestrians, and collision risk logs.
- **Azure Blob Storage** → Raw metadata files (nodes.csv, edges.csv, simulation outputs).

## Deployment Diagram :

## Component Diagram :


CLIENT_MACHINE
string Browser
string Streamlit_Dashboard
HTTP/HTTPS
WEB_SERVER
string Streamlit_App
API/Execution
APPLICATION_SERVER
string Python_Scripts
Inference
Queries/Logs
ML_SERVER
string YOLO
string GNN
string Risk_Scoring_Model
DATABASE_SERVER
string PostgreSQL
string Scenarios
string Vehicles
string Pedestrians
string Collision_Risks
Read/Write
STORAGE_NODE
string Azure_Blob_Storage
string Metadata_Files


<<Component>>
SimulationLayer
+SUMO
+CARLA
metadata, object data
<<Component>>
DetectionLayer
+YOLO
+GNN
intent predictions
<<Component>>
RiskAnalysisLayer
+RiskScoringModel
collision risk results
<<Component>>
APILogicLayer
+PythonScripts
+StreamlitIntegration
alerts, dashboards
store/retrieve scenarios & risks
<<Component>>
VisualizationLayer
+StreamlitDashboard
<<Component>>
DataLayer
+PostgreSQL
+AzureBlobStorage